Labo 13 — Labo pratique : la chaîne web → api dans le cluster

Objectif : déployer le front et l'API, voir nginx mourir faute de Service, créer le Service `api` et observer DNS, IP de cluster et EndpointSlices, exposer le front en NodePort puis en LoadBalancer, enfin passer par l'Ingress Traefik sur le port 80 — et vérifier que les endpoints suivent les Pods.

Prérequis — Labo 12 terminé : k3s, `localhost/api:1.0` dans le cluster, Podman avec `api:1.0`.

Fichiers fournis (`files/`) - `Dockerfile.web` , `nginx-proxy.conf` , `index.html` — l'image `web:1.0` : nginx + le front + le proxy `/api/` → `http://api:8080/` . - `deployment-api.yaml` , `deployment-web.yaml` — les deux Deployments (celui de l'API est celui du labo 12). - `service-api.yaml` (ClusterIP), `service-web.yaml` (NodePort 30080), `ingress-web.yaml` (Traefik, chemin `/`).

Étape 0 — L'image du front dans le cluster

```
mkdir -p ~/labo-docker/13 && cd ~/labo-docker/13 && cp <chemin-du-labo>/files/* .
podman build -q -f Dockerfile.web -t web:1.0 .
podman save localhost/web:1.0 | sudo k3s ctr images import -
sudo k3s crictl images | grep -E 'localhost/(api|web)'
```

Observez `localhost/web 1.0` à côté des `localhost/api` dans le cluster.

Étape 1 — Sans Service, pas de nom

```
kubectl create namespace lab13 ; kubectl config set-context --current --namespace=lab13
kubectl apply -f deployment-api.yaml -f deployment-web.yaml
kubectl rollout status deployment/api --timeout=60s
sleep 20 ; kubectl get pods
kubectl logs deploy/web --tail=1
```

Observez l'API `Running` et `web` en `CrashLoopBackOff` avec `RESTARTS 2` , et la cause : `nginx: [emerg] host not found in upstream "api" in /etc/nginx/conf.d/default.conf:15` .

Explication. Au labo 07, le réseau Podman nommait chaque conteneur. Ici, un Pod n'a pas de nom DNS : nginx résout `api` au démarrage, ne trouve rien, et sort. Seul un Service porte un nom.

Étape 2 — Le Service ClusterIP

```
kubectl apply -f service-api.yaml
kubectl rollout status deployment/web --timeout=120s
kubectl get pods
kubectl get svc api ; kubectl get endpointslices
```

Observez `web` passé `1/1 Running` **sans aucune commande de votre part** (`RESTARTS 3` , le backoff a fini par réussir) ; un Service `api ClusterIP 10.43.x.x <none> 8080/TCP` ; et un EndpointSlice `api-xxxxx` avec les deux IP des Pods de l'API.

```
kubectl exec deploy/web -- wget -qO- http://api:8080/actuator/health ; echo
kubectl exec deploy/web -- cat /etc/resolv.conf
kubectl exec deploy/web -- wget -qO- http://api.lab13.svc.cluster.local:8080/ ; echo
kubectl scale deployment api --replicas=3 ; sleep 8 ; kubectl get endpointslices
```

Observez `{"status":"UP"}` depuis le Pod `web` ; un `resolv.conf` avec `search lab13.svc.cluster.local svc.cluster.local cluster.local` et `nameserver 10.43.0.10` ; la même réponse par le nom complet ; et, après le passage à trois réplicas, **trois** adresses dans l'EndpointSlice.

Explication. Le Service a une IP virtuelle et un nom ; CoreDNS résout le nom, kube-proxy traduit l'IP vers l'un des endpoints, et la liste des endpoints suit les Pods prêts.

Étape 3 — NodePort : une porte sur le nœud

```
kubectl apply -f service-web.yaml ; sleep 3 ; kubectl get svc web
NODE=$(kubectl get nodes -o jsonpath='{.items[0].status.addresses[0].address}') ; echo $NODE
curl -s http://$NODE:30080/ | head -3
curl -s http://$NODE:30080/api/actuator/health ; echo
```

Observez `web NodePort 10.43.x.x <none> 80:30080/TCP`, l'IP de votre nœud (celle de la distribution WSL), le `<!doctype html>` du front et `{"status":"UP"}` via le proxy nginx — le tout sur le port `30080` du nœud. Depuis Windows, `http://localhost:30080/` ouvre la même page.

Explication. Un NodePort ouvre le même port sur chaque nœud et le relaie vers le Service. Pratique pour tester ; en production, on préfère une adresse sans port exotique.

Étape 4 — LoadBalancer : ServiceLB de k3s

```
kubectl expose deployment web --name=web-lb --type=LoadBalancer --port=8081 --target-port=80
sleep 10 ; kubectl get svc web-lb
kubectl get pods -n kube-system -l svccontroller.k3s.cattle.io/svcname=web-lb -o name
curl -s http://$NODE:8081/api/actuator/health ; echo
kubectl delete svc web-lb
```

Observez `web-lb LoadBalancer 10.43.x.x <IP-du-nœud> 8081:3xxxx/TCP` — l'`EXTERNAL-IP` est l'adresse du nœud —, un Pod `svclb-web-lb-…` dans `kube-system`, et `{"status":"UP"}` sur le port `8081`.

Explication. Kubernetes nu laisserait `<pending>` ; ServiceLB simule un équilibreur en ouvrant le port sur le nœud. Le port 80 était déjà pris par Traefik, d'où `8081`.

Étape 5 — L'Ingress

```
kubectl get svc -n kube-system traefik
kubectl apply -f ingress-web.yaml ; sleep 5 ; kubectl get ingress
curl -s http://localhost/ | head -3
curl -s http://localhost/api/actuator/health ; echo
curl -s http://$NODE/api/ ; echo
```

Observez Traefik en `LoadBalancer` sur `80:3xxxx/TCP,443:3xxxx/TCP` avec l'IP du nœud ; l'Ingress `web` de classe `traefik`, hôte `*`, `ADDRESS` = IP du nœud ; puis le front et l'API sur le **port 80**, sans NodePort : `http://localhost/`.

Explication. L'Ingress décrit « `/` → Service `web` » ; Traefik, déjà sur le port 80 du nœud, l'applique. Chaîne : port 80 → Traefik → Service `web` → nginx → (`/api/`) Service `api` → Pod. Depuis Windows, `http://localhost/` fonctionne — k3s tourne en root, la restriction des ports < 1024 ne joue pas.

Étape 6 — Les endpoints suivent les Pods

```
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
kubectl delete pod $POD ; sleep 1
kubectl get endpointslices -l kubernetes.io/service-name=api -o
jsonpath='{.items[0].endpoints[*].addresses[0]}' ; echo
kubectl port-forward svc/api 18080:8080 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/actuator/health ; echo ; kill %1
```

Observez trois adresses, puis, une seconde après la suppression d'un Pod, l'adresse de ce Pod remplacée par celle de son successeur (le ReplicaSet du labo 12 l'a recréé) ; et `port-forward` qui fonctionne aussi vers un **Service**.

Explication. Le Service ne connaît pas vos Pods par leur nom mais par leurs labels et leur état `Ready` : la liste se recalcule à chaque changement, sans intervention.

Nettoyage

```
kubectl delete namespace lab13
kubectl config set-context --current --namespace=default
kubectl get svc -A | grep -c lab13
```

Observez `0` : Services, Ingress et Deployments sont partis avec le namespace. Gardez `web:1.0` et `api:1.0` dans Podman et dans le cluster : les labos 14 et 15 complètent la stack avec la configuration et la base.

Ce que vous devez pouvoir affirmer maintenant

- Sans Service, nginx meurt : `host not found in upstream "api"` ; avec le Service, il repart seul au backoff suivant.
- Un Service = `ClusterIP 10.43.x.x`, un nom résolu par `10.43.0.10` via `search lab13.svc.cluster.local`, un EndpointSlice avec les IP des Pods prêts.
- `scale` à 3 = trois endpoints ; un Pod supprimé disparaît de la liste et son remplaçant y entre.
- NodePort `30080` sur l'IP du nœud ; LoadBalancer = ServiceLB, `EXTERNAL-IP` = IP du nœud, un Pod `svclb-…` par Service.
- Ingress `web` → Traefik sur le port 80 du nœud → `http://localhost/` et `http://localhost/api/actuator/health`.
- `port-forward svc/api` traverse tout sans rien exposer.